

Reviewer Pack — Minimal Rank of Brauer Group over Boolean Functions' Tensor ...

Entry 1c930eeff14d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 07:57:39 UTC

Minimal Rank of Brauer Group over Boolean Functions' Tensor Product Valuations

Entry ID: 1c930eeff14d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 07:57:39 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Brauer Groups) **Field B** (complexity object): Communication Complexity (Boolean Tensor Product)

Statement:

{‘quantitative_relation’: ‘For all boolean functions $f: \{0,1\}^n \rightarrow \{0,1\}$, the minimal rank of the Brauer group over the tensor product valuation of f is $\Theta(n^{2/3})$.’, ‘falsifier’: ‘There exists a boolean function f such that the minimal rank of its Brauer group over the tensor product valuation is $O(1)$.’}

Rationale (proposer’s reasoning):

{‘explanation’: “The Brauer group captures non-trivial linear structures in fields, which are relevant to understanding non-commutative aspects. By linking this with boolean functions’ tensor products, we could uncover a new dimension of complexity theory.”, ‘connection_to_complexity’: “Boolean functions’ tensor products are a natural representation for studying communication complexity. The Brauer group’s rank might reflect the inherent difficulty in distributing information among parties.”}

Taxonomy category: Representation Theory (Brauer Groups) × Communication Complexity (Boolean Tensor Product) (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dc453ac10abc7818

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of randomly generated boolean functions have a Brauer group minimal rank that is within a factor of 3 from $n^{2/3}$, and all seeds show a Spearman’s rank correlation coefficient greater than or equal to 0.7 with $n^{2/3}$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Rank Brauer Group Boolean Functions Tensor Product Valuation - Representation Theory Brauer Groups Communication Complexity Boolean Tensor Products - Quantitative Relation $n^{(2/3)}$ Minimal Rank Brauer Group Boolean Function Tensor Product Valuation

Top relevant hits considered: - [<http://arxiv.org/abs/1405.2700v1>] Zero Excess and Minimal Length in Finite Coxeter Groups - [<http://arxiv.org/abs/1110.6618v2>] Brauer relations in finite groups II - quasi-elementary groups of order $p^a q$ - [<http://arxiv.org/abs/math/0503746v2>] Qd(p)-free rank two finite groups act freely on a homotopy product of two spheres - [<http://arxiv.org/abs/1603.02493v4>] Clifford theory for glider representations - [<http://arxiv.org/abs/1001.0462v2>] Representation Theory of Finite Groups - [<http://arxiv.org/abs/0807.3095v1>] Guide to the Atlas Software: Computational Representation Theory of Real Reductive Groups - [<http://arxiv.org/abs/2510.17487v1>] Directional Search for Persistent Gravitational Waves: Results from the First Part of LIGO-Virgo-KAGRA's Fourth Observin - [<http://arxiv.org/abs/1411.4413v2>] Observation of the rare $B_s^0 \rightarrow \mu^+ \mu^-$ decay from the combined analysis of CMS and LHCb data

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=-9, elapsed=98.0s

5.1 Generated Python source

```
import random
import math

def generate_boolean_function(n):
    return [random.randint(0, 1) for _ in range(2**n)]

def tensor_product(f, g):
    n = int(math.log2(len(f)))
    m = int(math.log2(len(g)))
    result = []
    for x in range(2**n):
        for y in range(2**m):
            result.append(f[x] * g[y])
    return result

def compute_brauer_group_rank(f, g):
    tensor_valuation = tensor_product(f, g)
    n = int(math.log2(len(tensor_valuation)))
```

```

rank = 0
for i in range(n):
    if all(tensor_valuation[j] == 0 for j in range(2**n) if (j & (1 << i)) != 0):
        rank += 1
return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    f = generate_boolean_function(n)
    g = generate_boolean_function(n)
    rank = compute_brauer_group_rank(f, g)
    expected_rank = n ** (2/3)
    return {
        "metric_name": "Brauer group rank",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": abs(rank - expected_rank) <= 3 * expected_rank,
        "counterexample": "" if conjecture_holds else f"n={n}, rank={rank}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 7 for i in range(5, 8)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...{result}...}}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_rank = math.sqrt(sum((r["metric_value"] - mean_rank)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"n={results[0]['instances_tested']}, rank={result}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means the pre-registered support condition could not be unambiguously met. | next: Re-run the test to ensure it completes successfully and meets the pre-registered criteria.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12448
2	propose	ollama_remote	glm4:latest	0	0	10030
3	preregistration	ollama_remote	glm4:latest	0	0	5715
4	novelty	ollama_remote	glm4:latest	0	0	4726
5	novelty	ollama_remote	glm4:latest	0	0	6046
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18821
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8403
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12190
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7854
10	judge	ollama_remote	glm4:latest	0	0	61115

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 147347 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/1c930eeff14d.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/1c930eeff14d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/1c930eeff14d.tar.gz (if generated)